Course Intro / Relational DB Review

ISYS2099 | ISYS3434 · Database Applications · Week 1

Tom Huynh
School of Science, Engineering & Technology
RMIT University Vietnam



RMIT UNIVERSITY

Today's plan

01

Course Overview

How the course runs, assessments and how to succeed

≈ 15 min

02

Data Modeling & ER Diagrams

Entities, relationships and reading the tutorial diagram

≈ 20 min

03

Relational Model & Normalization

Keys, mapping ER to tables, and 1NF → 2NF → 3NF

≈ 30 min

04

SQL Review

DDL, DML, joins, grouping and four query patterns

≈ 25 min

Then the tutorial (1.5 h): install MySQL, load the sample COMPANY database, write 12 SQL queries and normalize an Order table.



01

Course Overview

Part 01

Why databases | Roadmap | Assessments | How to succeed

RMIT UNIVERSITY

Meet your lecturer



Tom Huynh

Course Coordinator & Lecturer

tom.huynh@rmit.edu.vn

Saigon South · Room 2.4.27

Email to book an appointment

Background

Bachelor of Computer Science
University of Melbourne

Master of Artificial Intelligence
University of New South Wales (UNSW)

Head of Web Development
Weboptimizers, Melbourne

Director of Machine Learning
CoderSchool, Vietnam: data science bootcamps

Lecturer
International University (VNU), now RMIT Vietnam

Passionate about **data science & machine learning**

Databases sit behind every app you use



Banking

Accounts, balances, transfers

E-commerce

Products, carts, orders

Ride-hailing

Drivers, trips, locations

Social media

Users, posts, likes

Streaming

Catalogue, watch history

University (Canvas)

Students, courses, grades

This course: make databases **fast** (indexing, optimisation, partitioning), **programmable** (procedures, triggers), **reliable** (transactions) and **secure**, in MySQL and MongoDB.

By the end of this course you will be able to…



CLO 1

Design

Apply advanced data analysis & modelling, physical design, integrity, security and transaction management



CLO 2

Program

Create triggers, stored procedures and functions to make a database more usable



CLO 3

Protect

Apply techniques to store, access, secure and recover data efficiently



CLO 4

Optimise

Build an efficient database application: storage management, indexing, query optimisation



CLO 5

Go NoSQL

Use MongoDB to store and retrieve non-relational data



CLO 6

Build the interface

Develop a simple web-based interface for a database

Your 12-week journey

Relational databases · MySQL

1	2	3	4	5	6
Course intro & relational review	**Indexing**	**Query optimisation**	**Partitioning**	**Views, procedures, functions, triggers**	**Transactions & concurrency**
You are here					A1 Midterm · 25%

NoSQL · MongoDB, then security & review

7	8	9	10	11	12
Intro to NoSQL	**Aggregations & joins**	**Optimising MongoDB**	Holidays: self-study, compare a DBMS	**Database security**	**Review**
A2 + A3 released				A2 + A3 due	A4 Final · 35%

A1 Midterm test 25% · **A2** Project prototype 25% · **A3** Project presentation 15% · **A4** Final test 35%

How each week works

Before class

- Read the slides (≈ 1 h)
- Note what is unclear

In class · 3 h

- 1.5 h lecture: concepts, demos, activities
- 1.5 h tutorial: hands-on practice

After class

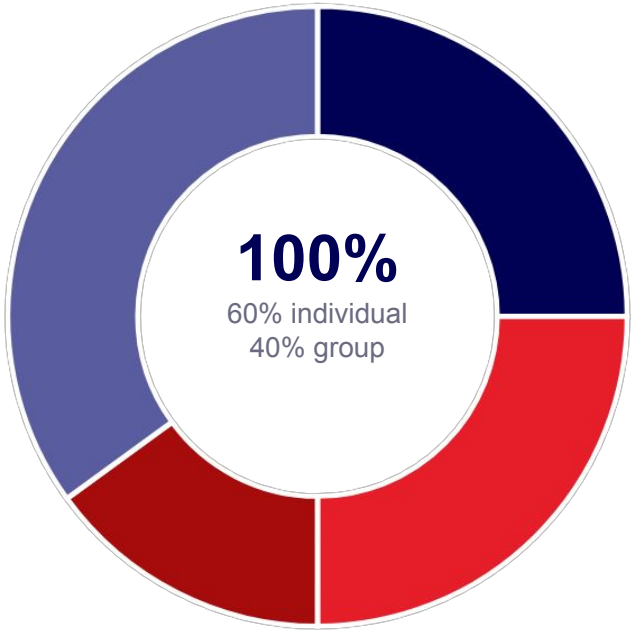
- Finish the exercises (≈ 1 h)
- Ask in the weekly discussion
- From W6: work on the group project

3 h in class every week (except Week 10)

≤ 66 h self-study outside class, in total

48 h max reply time on the course forum

Assessment at a glance



25%

A1 · Midterm test

Week 6 · in class · individual · limited resource access

25%

A2 · Group project prototype

Released Week 7 · due Week 11 · groups of 5

15%

A3 · Group project presentation

Released Week 7 · due Week 11 · groups of 5

35%

A4 · Final test

Week 12 · in class · individual · limited resource access

Assessment rules and academic integrity

In-class tests

A1 Midterm · A4 Final

- **No AI tools**
- **No collaboration**
- **Limited resources only**
- **Screen recording + web camera**

Group project

A2 Prototype · A3 Presentation

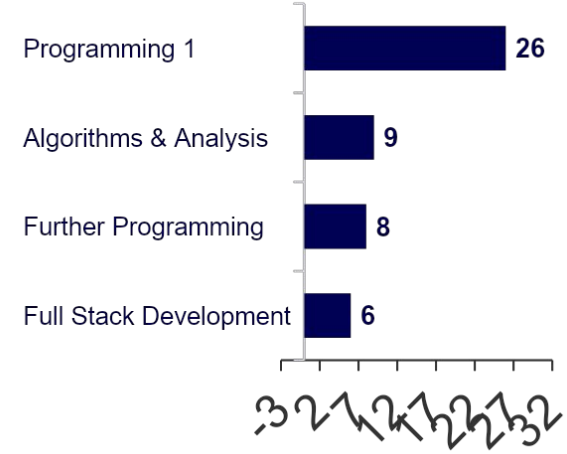
- **AI tools allowed…**
- **…documented clearly & explicitly**

Example statement

"We used ChatGPT to draft test data; we checked and edited all SQL ourselves."

Students at misconduct hearings

recent offerings of other courses



Do your own work, cite others' ideas.
Written work is checked by Turnitin.

How to succeed and where to get help

Habits that work

Be an active learner

Class is a discussion, not just a lecture

Ask questions

You will learn quicker this way

Don't memorise

Understand why a query works

Challenge yourself

Try the (*) questions

Where to get help

- **Course Q&A forum** replies within 48 h
- **Email your lecturer** to book a meeting
- **Student Academic Success** advisors, tutors, workshops
- **Library** databases, referencing help
- **Equitable Learning & Accessibility** learning plans, adjustments
- **Counselling (CAPS)** confidential support
- **Student Connect** enrolment, timetables, ID cards



02

Data Modeling & ER Diagrams

Part 02

Building blocks | Attribute types | Relationships | Entity types | Tutorial diagram

RMIT UNIVERSITY

From the real world to a running database

1 Requirements → 2 Conceptual → 3 Logical → 4 Physical

What must we store, and for whom?

Example

“A company tracks employees, the departments they work in and the projects they work on.”

ER diagram: entities, attributes, relationships

Example



Relational schema: tables, keys, foreign keys (normalized)

Example

```
EMPLOYEE(ssn, name, dept)

WORKS(ssn, project, hours)
```

SQL DDL + storage design: indexes, partitions

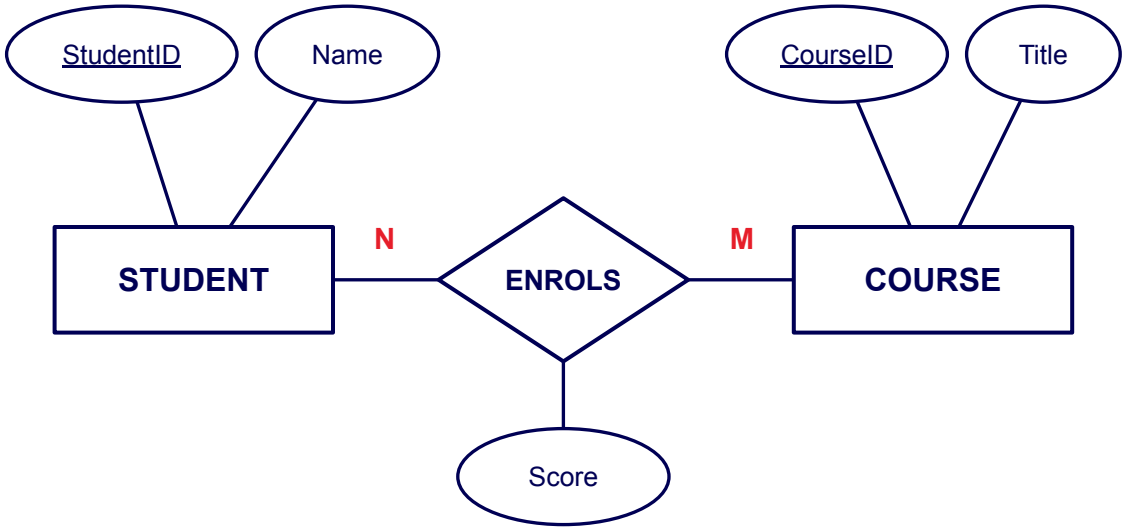
Example

```
CREATE TABLE ...
CREATE INDEX ...
PARTITION BY ...
```

Today: review steps 2 and 3

Weeks 2 to 4: make it fast

Three building blocks of an ER model



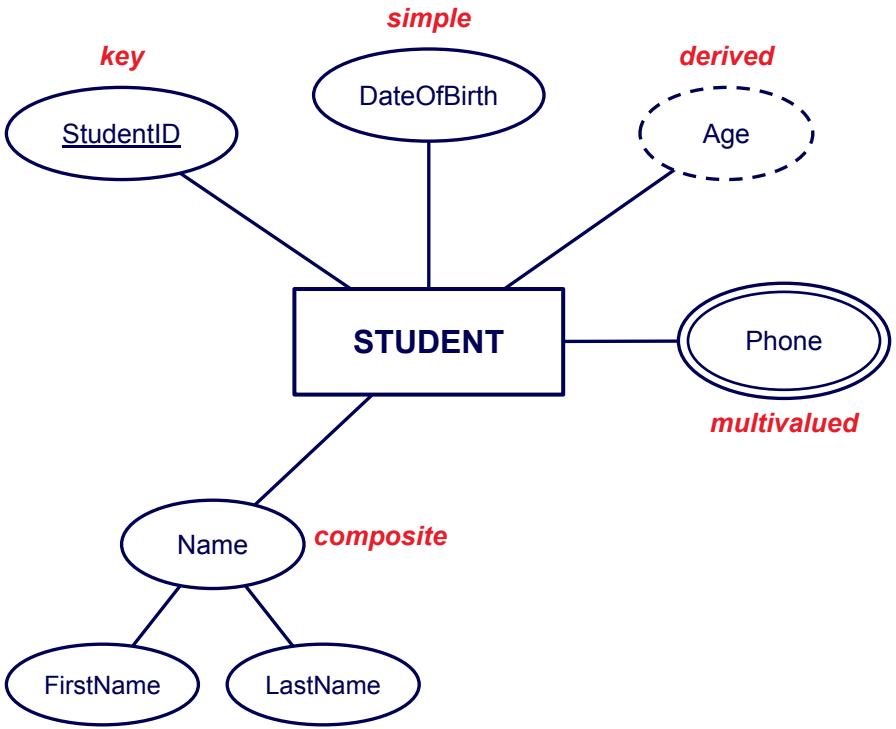
Entity
A real-world object or concept we store data about
e.g. Student, Course, Book

Attribute
A property or characteristic of an entity
e.g. StudentID, Name, DateOfBirth

Relationship
A logical association between entities
e.g. a Student enrols in a Course

Data modeling = defining data elements and their relationships to represent a real-world scenario. The **ER (Entity-Relationship) model** is our design tool.

Attribute types

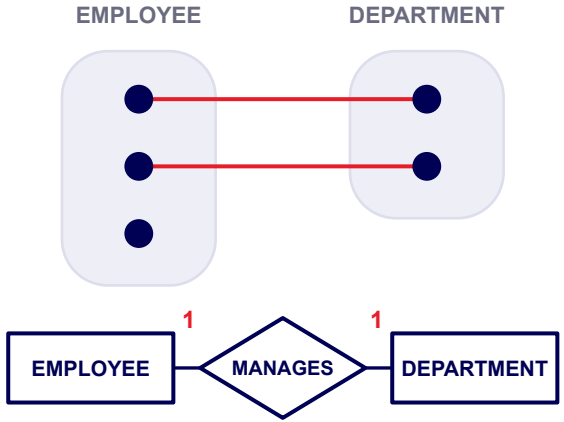


Type	Rule	Example
Simple	Cannot be divided further	DateOfBirth
Composite	Made of smaller sub-parts	Name = First + Last
Derived	Computed from other attributes	Age (from DateOfBirth)
Multivalued	Can hold several values	Phone numbers
Key	Uniquely identifies each entity	StudentID

Your turn · 2 min · answer in the chat
List simple, composite, derived and multivalued attributes of an RMIT student.

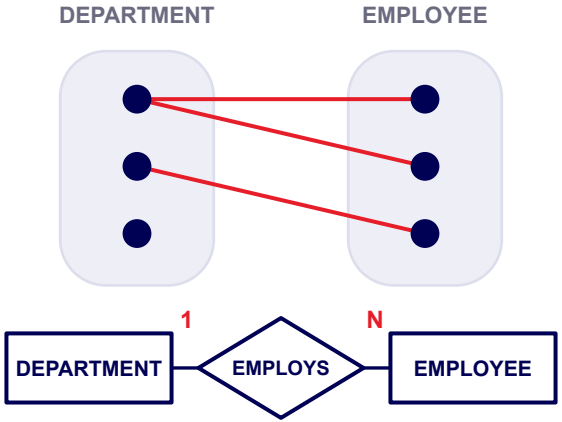
Relationship cardinality: 1:1, 1:N, M:N

1 : 1 One-to-one



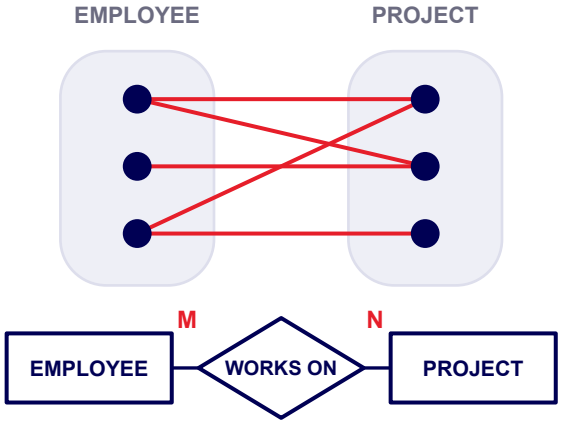
An employee manages at most one department; a department has exactly one manager.

1 : N One-to-many



A department employs many employees; each employee works for one department.

M : N Many-to-many



An employee works on many projects; a project has many employees.

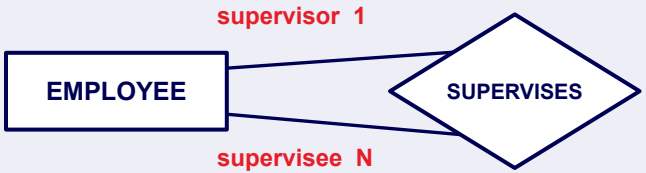
Degree and participation

Degree: how many entity types take part

Binary



Unary (recursive)



Participation: must every entity take part?



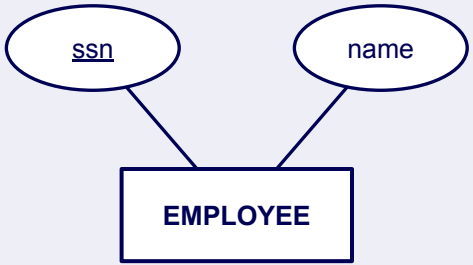
Partial · single line
Most employees manage nothing

Total · double line
Every department has a manager

Your turn · 3 min Sketch a shopping-cart app (Customer, Cart, Product, Order…). In the chat: which relationships are 1:1, 1:N or M:N? Which participations are total?

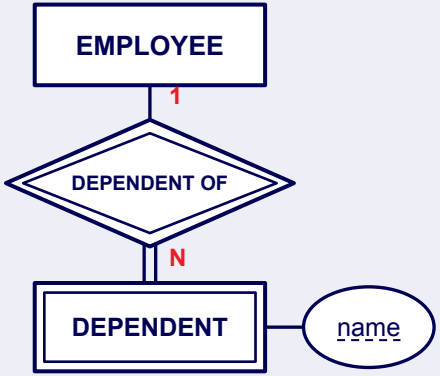
Entity types: regular, weak, associative

Regular (strong)



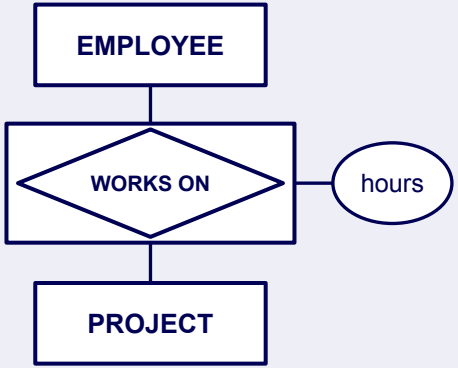
Has its own key (ssn) and exists independently of other entities.

Weak



No key of its own: identified by its owner's key + a partial key (ssn + name). A dependent only exists for an employee.

Associative



Represents an M:N relationship as an entity, so it can carry its own attributes (hours worked).

ER notation cheat sheet

Chen notation: used in the tutorial diagram



Entity



Weak entity



Attribute



Key attribute



Partial key (weak entity)



Multivalued attribute



Derived attribute



Relationship

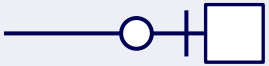


Identifying relationship



Total vs partial participation

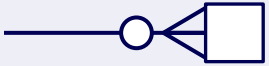
Crow's foot: common in tools



Optional one
0..1



Mandatory one
1..1



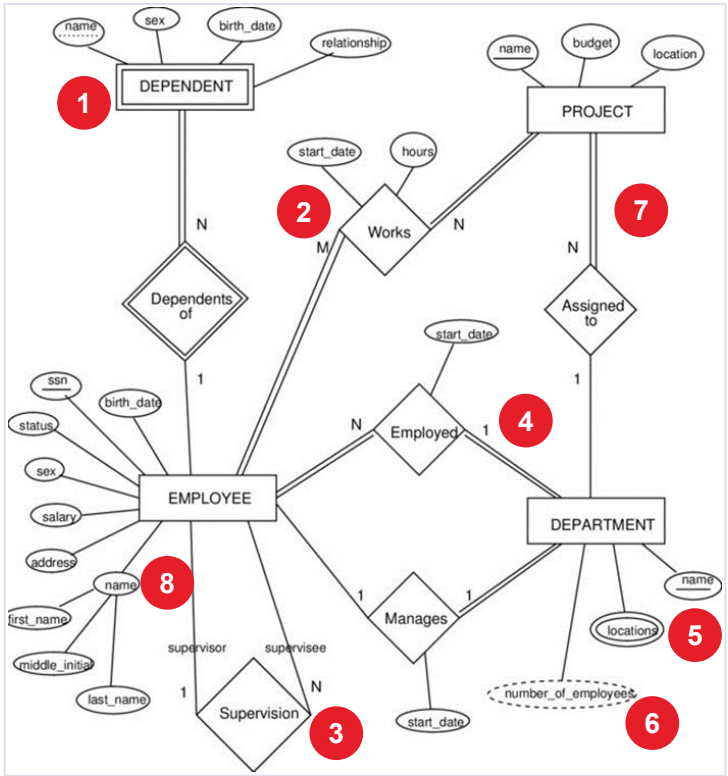
Optional many
0..N



Mandatory many
1..N

Symbol next to the entity = maximum; inner symbol = minimum.

Decode the tutorial's Chen ER diagram



1. **DEPENDENT: weak entity**
   double box; key = employee ssn + partial key name
2. **Works: M:N relationship**
   has its own attributes (start_date, hours) → own table
3. **Supervision: recursive 1:N**
   one supervisor, many supervisees
4. **Employed: 1:N**
   each employee works in one department
5. **locations: multivalued**
   double ellipse → separate table
6. **number_of_employees: derived**
   dashed ellipse → compute with COUNT, don't store
7. **Double lines: total participation**
   e.g. every project is assigned to a department
8. **name: composite**
   first_name, middle_initial, last_name



03

Relational Model & Normalization

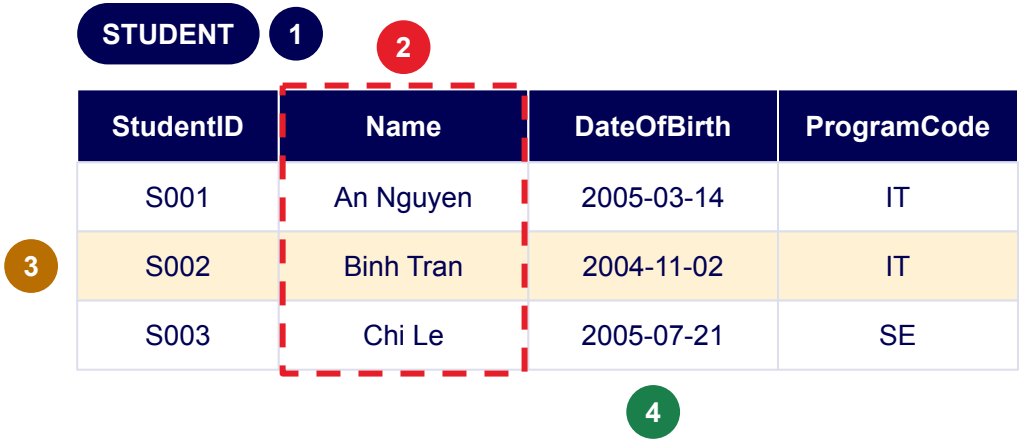
Part 03

Relations & keys | ER → tables | Anomalies | 1NF · 2NF · 3NF | Trade-offs

RMIT UNIVERSITY

Anatomy of a relation

*A relational database stores data in **relations** (tables) made of rows and columns.*



1. **Relation**
   a table with a unique name
2. **Attribute**
   a column, i.e. a property of the entity
3. **Tuple**
   a row, i.e. one record
4. **Domain**
   the set of valid values, e.g. DateOfBirth ∈ valid dates

Schema: the structure (design time)
`STUDENT(StudentID, Name, DateOfBirth, ProgramCode)`

Instance: the actual rows at one moment
changes with every INSERT, UPDATE, DELETE

Keys: how rows are identified and linked

PK CK FK

StudentID	Email	Name	ProgramCode
S001	an.nguyen@rmit.edu.vn	An Nguyen	IT
S002	binh.tran@rmit.edu.vn	Binh Tran	IT
S003	chi.le@rmit.edu.vn	Chi Le	SE

STUDENT

references

PK

ProgramCode	ProgramName
IT	Information Technology
SE	Software Engineering

PROGRAM

CK Candidate key

Any minimal set of attributes that uniquely identifies a row. A table can have several: StudentID, Email.

PK Primary key

The candidate key we choose as the main identifier. Unique and never NULL.

FK Foreign key

References another table's primary key. Links tables and enforces referential integrity.

ER → relational: six mapping rules

1 Regular entity



Create a table with its attributes. The key becomes the primary key.

2 Weak entity



Table with its attributes + an FK to the owner. PK = (owner key, partial key).

3 1:1 or 1:N



No new table: add an FK on the "many" side (for 1:1, on the total-participation side).

4 M:N relationship



New table with FKs to both entities (together the PK) + the relationship's attributes.

5 Multivalued attribute



New table: (entity key, value); both columns form the PK.

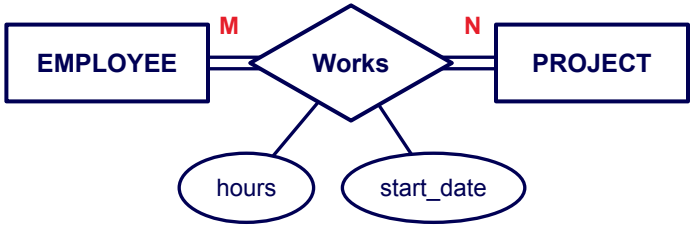
6 Derived attribute



Usually not stored. Compute it in a query when needed.

Mapping in action: Works and Employed

M:N → new table



EMPLOYEE

ssn
first_name
…

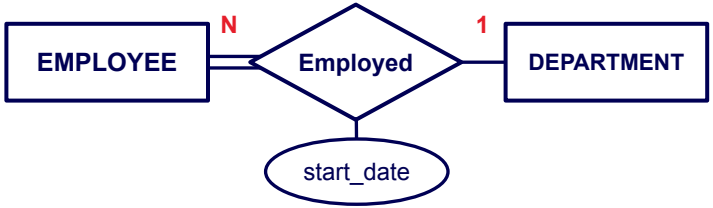
WORKS

ssn
project_name
start_date
hours

PROJECT

name
budget
location

1:N → foreign key on the N side



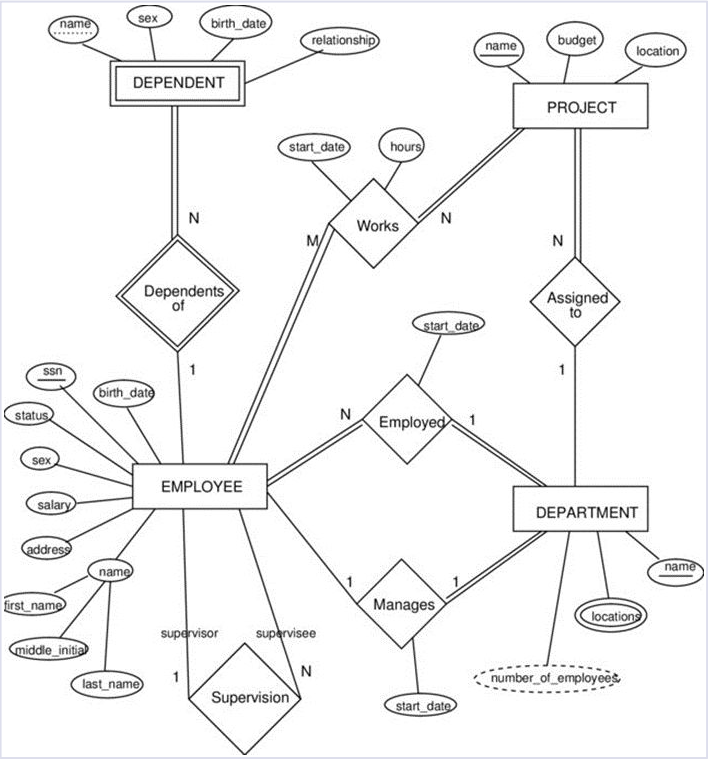
EMPLOYEE

ssn
first_name
…
dept_name
dept_start_date

DEPARTMENT

name
manager_ssn
…

Your turn: map the tutorial ER diagram



On your own · 6 min apply the six rules in order

1. **One table per strong entity** EMPLOYEE, DEPARTMENT, PROJECT
2. **Weak entity → table with the owner's key** DEPENDENT
3. **1:1 and 1:N → foreign keys** Manages, Employed, Assigned to, Supervision
4. **M:N → new table** Works
5. **Multivalued → new table** locations
6. **Derived → don't store** number_of_employees

Post in the chat: how many tables? What is each primary key?

One possible mapping of the COMPANY diagram

EMPLOYEE

ssn
first_name
middle_initial
last_name
birth_date
sex
status
salary
address
supervisor_ssn → EMPLOYEE
dept_name → DEPARTMENT
dept_start_date

The Sample DB script uses its own names and keys: compare on the next slide.

DEPARTMENT

name
manager_ssn → EMPLOYEE
manager_start_date

DEPT_LOCATION

dept_name → DEPARTMENT
location

DEPENDENT

ssn → EMPLOYEE
name
sex
birth_date
relationship

PROJECT

name
budget
location
dept_name → DEPARTMENT

WORKS

ssn → EMPLOYEE
project_name → PROJECT
start_date
hours

PK bold + underlined
FK red, → referenced table
number_of_employees is derived → not stored

Check your mapping against the real Sample DB

EMPLOYEE

Ssn
Fname
Minit
Lname
Bdate
Address
Sex
Salary
Super_ssn → EMPLOYEE
Dno → DEPARTMENT

Same 6 tables as our mapping
Not in the script: status, budget, start_date of Works and Employed
Query with these names: SHOW TABLES; DESCRIBE employee;

DEPARTMENT

Dnumber
Dname UNIQUE
Mgr_ssn → EMPLOYEE
Mgr_start_date

DEPT_LOCATIONS

Dnumber → DEPARTMENT
Dlocation

DEPENDENT

Essn → EMPLOYEE
Dependent_name
Sex
Bdate
Relationship

PROJECT

Pnumber
Pname UNIQUE
Plocation
Dnum → DEPARTMENT

WORKS_ON

Essn → EMPLOYEE
Pno → PROJECT
Hours

PK bold + underlined
FK red, → referenced table
amber: a number key (or an FK to one) where we used a name

Extra practice: map the bank ER diagram

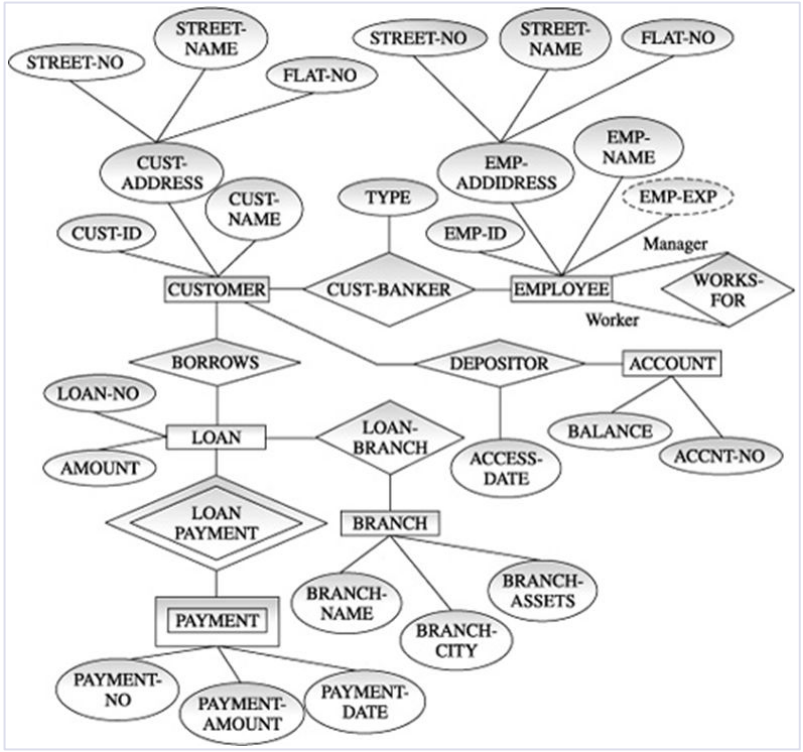


Image source: oreilly.com/library/view/database-systems-concepts/9788177585674

Self-study

Map this diagram into a relational schema.

- CUST-ADDRESS, EMP-ADDRESS are composite → flatten into simple columns
- EMP-EXP is derived → don't store it
- PAYMENT is weak: identified through LOAN PAYMENT
- WORKS-FOR is recursive (Manager / Worker)
- Cardinalities aren't shown, so write down your assumptions first

04

Normalization

RMIT UNIVERSITY

Why normalize? Spot the anomalies

Normalization organizes data to minimize redundancy and protect data integrity.

StudentID	StudentName	Prog	ProgramName	CourseID	CourseName	Score
S001	An Nguyen	IT	Information Technology	ISYS2099	Database Applications	75
S001	An Nguyen	IT	Information Technology	COSC2469	Algorithms & Analysis	68
S002	Binh Tran	IT	Information Technology	ISYS2099	Database Applications	82
S003	Chi Le	SE	Software Engineering	ISYS2099	Database Applications	90

■ *same fact stored again and again = redundancy* · *Prog = ProgramCode*

Why two key columns?

One row = one student in one course.

StudentID repeats (S001). CourseID repeats (ISYS2099).

Only the pair is unique.

Insertion anomaly

Can't add a new course (e.g. ISYS3434) until a student enrols: StudentID is part of the key, so it can't be NULL.

Update anomaly

Rename "Database Applications" → update every row. Miss one = inconsistent data.

Deletion anomaly

Delete S003's only enrolment → we also lose that SE = Software Engineering.

What is normalization? 1NF, 2NF, 3NF

Normalization = splitting a table into smaller tables so that each fact is stored once.

Why: a fact stored once can't repeat or contradict itself, so the three anomalies go away.

1NF — One value per cell

Rule: every cell holds one value. No lists or repeating groups.

Why: a value hidden in a list can't be searched, counted or checked.

Example: a student's list of courses becomes one row per course.

2NF — The whole key

Rule: 1NF + every non-key attribute depends on the whole key.

Why: facts that need only part of the key repeat in every row.

Example: StudentName needs only StudentID, so it moves to STUDENT.

3NF — Nothing but the key

Rule: 2NF + non-key attributes don't depend on each other.

Why: facts about something else repeat in many rows.

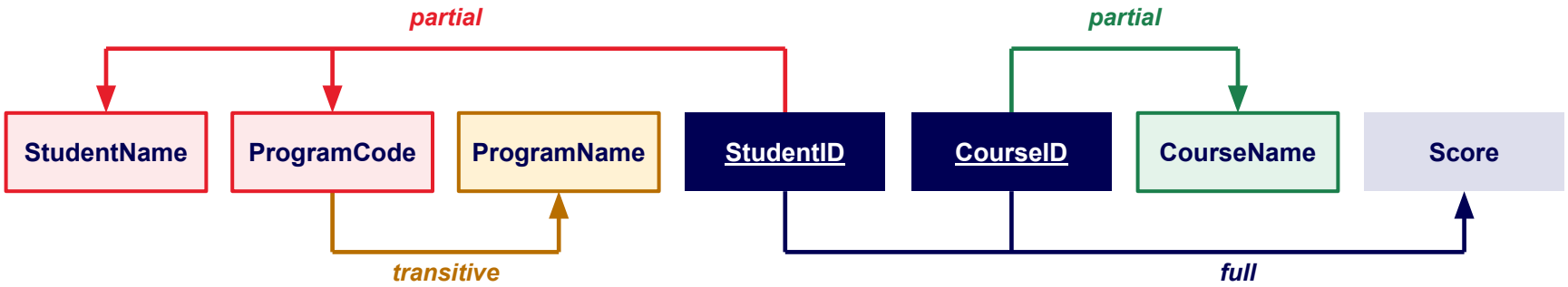
Example: ProgramName depends on ProgramCode, so it moves to PROGRAM.



Next: functional dependencies, the tool for checking what each attribute depends on.

Functional dependencies

X → **Y** "X determines Y": if two rows agree on X, they must agree on Y. e.g. **`CourseID`** → **`CourseName`**



Full dependency

on the whole key: (StudentID, CourseID) → Score ✔

Partial dependency

on part of a composite key ✘ violates 2NF

Transitive dependency

on a non-key attribute ✘ violates 3NF

Step 1 → 1NF: make every value atomic

Before · UNF

StudentID	StudentName	Prog	ProgramName	Courses (CourseID · CourseName · Score)
S001	An Nguyen	IT	Information Technology	ISYS2099 · Database Applications · 75 COSC2469 · Algorithms & Analysis · 68
S002	Binh Tran	IT	Information Technology	ISYS2099 · Database Applications · 82
S003	Chi Le	SE	Software Engineering	ISYS2099 · Database Applications · 90

Split the list: one row per student per course

After · 1NF

StudentID	StudentName	Prog	ProgramName	CourseID	CourseName	Score
S001	An Nguyen	IT	Information Technology	**ISYS2099**	Database Applications	75
S001	An Nguyen	IT	Information Technology	**COSC2469**	Algorithms & Analysis	68
S002	Binh Tran	IT	Information Technology	**ISYS2099**	Database Applications	82
S003	Chi Le	SE	Software Engineering	**ISYS2099**	Database Applications	90

1NF rule: every cell holds one atomic value: no lists, no repeating groups. **New key: (StudentID, CourseID)**

Step 2 → 2NF: remove partial dependencies

StudentID	CourseID	StudentName	ProgramCode	ProgramName	CourseName	Score

StudentName, ProgramCode, ProgramName depend only on StudentID; **CourseName** only on CourseID → split by determinant



STUDENT

StudentID	StudentName	Prog	ProgramName
S001	An Nguyen	IT	Information Technology
S002	Binh Tran	IT	Information Technology
S003	Chi Le	SE	Software Engineering

COURSE

CourseID	CourseName
ISYS2099	Database Applications
COSC2469	Algorithms & Analysis

ENROLLMENT

StudentID	CourseID	Score
S001	ISYS2099	75
S001	COSC2469	68
S002	ISYS2099	82
S003	ISYS2099	90

2NF = 1NF + every non-key attribute depends on the **whole** key. (Only a composite key can break it.)

Still repeated: "Information Technology" → 3NF

Step 3 → 3NF: remove transitive dependencies



ProgramName depends on the key only **through** ProgramCode

STUDENT (2NF)

StudentID	StudentName	Prog	ProgramName
S001	An Nguyen	IT	Information Technology
S002	Binh Tran	IT	Information Technology
S003	Chi Le	SE	Software Engineering

STUDENT

StudentID	StudentName	Prog
S001	An Nguyen	IT
S002	Binh Tran	IT
S003	Chi Le	SE

PROGRAM

Prog	ProgramName
IT	Information Technology
SE	Software Engineering

Prog (ProgramCode) in STUDENT is now a foreign key → PROGRAM

Final 3NF schema: 4 tables

STUDENT(**StudentID**, StudentName, ProgramCode)

PROGRAM(**ProgramCode**, ProgramName)

COURSE(**CourseID**, CourseName)

ENROLLMENT(**StudentID, CourseID**, Score)

3NF = 2NF + no transitive dependencies: non-key attributes depend on **nothing but the key**.

The normalization staircase

> *Every non-key attribute must depend on **the key, the whole key, and nothing but the key.***
>
> After William Kent (1983)

UNF

Lists and repeating groups in cells

"the key"

1NF

Atomic values, one fact per row

"the whole key"

2NF

No partial dependencies

"nothing but the key"

3NF

No transitive dependencies

Tutorial bridge: Exercise 2, normalize Order

```
Order (OrderID, CustomerID, CustomerName, CustomerPhone, Products, OrderDate)
```

One customer per order · many products per order · each product has a ProductID and a ProductName

1NF Ask yourself

- Which attribute is not atomic?
- Give each product its own row.
- What is the new primary key?

2NF Ask yourself

- Which attributes depend on OrderID alone?
- Which depend on ProductID alone?
- Move each group to its own table.

3NF Ask yourself

- Which attributes describe the customer, not the order?
- CustomerID → … ?
- Which foreign keys link the tables?

Self-check: each final table should describe exactly one thing: a customer, an order, a product, or a line on an order.

Normalize or denormalize? It's a trade-off



Normalize when…

- Data consistency is critical
- You need to eliminate redundancy
- The system has frequent inserts and updates
- Integrity and a clear logical design matter
- You are in the initial design phase





Denormalize when…

- The system is read-heavy
- Query speed matters more than update speed
- Reports and analytics need fewer joins
- Datasets are small and redundancy is cheap
- Tables are rarely updated

Normalized: fewer anomalies, more JOINs · **Denormalized:** faster reads, copies to sync · **Weeks 2 to 4: make it fast**

04

SQL Review

Part 04

Command families | DDL & DML | SELECT pipeline | Joins | Query patterns

RMIT UNIVERSITY

SQL in one picture: four command families

DDL
Data Definition

Define structure

`CREATE · ALTER · DROP`

DML
Data Manipulation

Work with data

`SELECT · INSERT · UPDATE · DELETE`

DCL
Data Control

Control access

`GRANT · REVOKE`

→ Week 11

TCL
Transaction Control

Group changes safely

`COMMIT · ROLLBACK`

→ Week 6

Syntax basics

- Statements end with `;`
- Keywords are case-insensitive; UPPERCASE by convention
- Text values in single quotes: `'An Nguyen'`

Common data types

`INT`	`42`	`DECIMAL(p,s)`	`1234.50`
`VARCHAR(n)`	`'IT'`	`BOOLEAN`	`TRUE`
`DATE`	`'2026-10-20'`		

CREATE TABLE: turning the schema into SQL

```sql
CREATE TABLE Program (
  ProgramCode VARCHAR(10)  PRIMARY KEY,        -- 1
  ProgramName VARCHAR(100) NOT NULL            -- 2
);
CREATE TABLE Student (
  StudentID   VARCHAR(10)  PRIMARY KEY,
  Name        VARCHAR(100) NOT NULL,
  Email       VARCHAR(100) UNIQUE,             -- 3
  ProgramCode VARCHAR(10),
  FOREIGN KEY (ProgramCode)                    -- 4
    REFERENCES Program(ProgramCode)
);
```

1. **`PRIMARY KEY`**
   unique + never NULL: the row's identity
2. **`NOT NULL`**
   a value is required
3. **`UNIQUE`**
   no duplicates; with NOT NULL it's a candidate key
4. **`FOREIGN KEY ... REFERENCES`**
   value must exist in Program (referential integrity)



Create parent tables first (Program before Student). A foreign key can only reference a table that already exists.

Change the structure vs change the data

Change the structure DDL

Add a column

```
ALTER TABLE Student ADD Phone VARCHAR(20);
```

Drop a column

```
ALTER TABLE Student DROP COLUMN Phone;
```

Change a column

```
ALTER TABLE Student MODIFY Name VARCHAR(150);
```

Change the data DML

Insert rows

```
INSERT INTO Student (StudentID, Name)
VALUES ('S004', 'Dung Pham');
```

Insert the result of a query

```
INSERT INTO Student (StudentID, Name)
SELECT ApplicantID, Name FROM Applicant;
```

Update

```
UPDATE Student SET Name = 'Bob'
WHERE StudentID = 'S001';
```

Delete

```
DELETE FROM Student WHERE StudentID = 'S003';
```



UPDATE or DELETE without WHERE changes every row. Run the matching SELECT … WHERE first.

How MySQL reads a SELECT: the logical order

```
SELECT   CourseID, AVG(Score) AS avg_score   5
FROM     Enrollment                          1
WHERE    Score IS NOT NULL                   2
GROUP BY CourseID                            3
HAVING   COUNT(*) >= 10                      4
ORDER BY avg_score DESC                      6
LIMIT    3;                                  7
```

Why it matters WHERE runs before GROUP BY and SELECT, so **`COUNT(*)`** and the alias **`avg_score`** don't exist yet. Filter groups with HAVING.

Week 3 preview the optimizer may run steps in another physical order, but the result must match this logical order.

1. **`FROM / JOIN`** pick and combine tables
2. **`WHERE`** filter rows
3. **`GROUP BY`** form groups
4. **`HAVING`** filter groups
5. **`SELECT`** compute output columns
6. **`ORDER BY`** sort
7. **`LIMIT`** keep the first n rows

Filter, sort and limit

Filter rows `WHERE`

```
SELECT StudentID, Name FROM Student;
SELECT * FROM Student
WHERE ProgramCode = 'IT';
SELECT StudentID AS Code,
       Name AS `Friendly Name`
FROM Student;
```

`=` `!=` `<` `<=` `>` `>=` `AND` `OR` `NOT`

`BETWEEN` `IN` `LIKE` `IS NULL` `IS NOT NULL`

NULL = unknown. Write `IS NULL`, never `= NULL` (it is never true).

Sort & limit `ORDER BY · LIMIT`

```
SELECT * FROM Result ORDER BY Score DESC;
SELECT * FROM Student
ORDER BY ProgramCode ASC, Name ASC;
SELECT * FROM Student ORDER BY StudentID
LIMIT 5 OFFSET 10;  -- skip 10, return 5
```



Pagination: page p, n rows per page
`LIMIT n OFFSET (p-1)*n`

GROUP BY and HAVING, step by step

1 Rows

StudentID	Score
S001	75
S001	68
S001	45
S002	82
S003	90
S003	71
S004	40

2 `WHERE Score >= 50`

StudentID	Score
S001	75
S001	68
~~S001~~	~~45~~
S002	82
S003	90
S003	71
~~S004~~	~~40~~

3 `GROUP BY StudentID`

S001	75 · 68	2
S002	82	1
S003	90 · 71	2

`COUNT(*)` per group

4 `HAVING COUNT(*) >= 2`

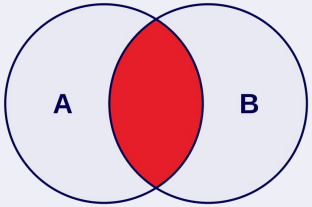
StudentID	completed
S001	2
S003	2

~~S002 (1)~~ *removed*

```sql
SELECT StudentID, COUNT(*) AS completed
FROM Enrollment
WHERE Score >= 50          -- filters rows
GROUP BY StudentID
HAVING COUNT(*) >= 2;      -- filters groups
```

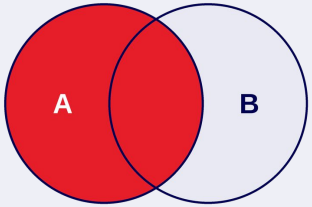
COUNT · SUM · AVG · MIN · MAX
Every SELECT column must be in GROUP BY or inside an aggregate.

JOINs: which rows survive?



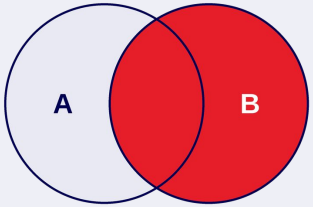
INNER JOIN

Only rows that match in both tables



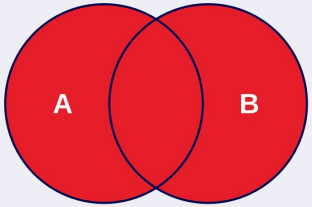
LEFT JOIN

All rows of A + matches from B (NULL if none)



RIGHT JOIN

All rows of B + matches from A (NULL if none)



FULL JOIN

All rows of both tables (not built into MySQL)

```sql
SELECT s.Name AS Student, p.ProgramName
FROM Student s
LEFT JOIN Program p
  ON s.ProgramCode = p.ProgramCode;
-- students with no program still appear
```

MySQL has no FULL OUTER JOIN
Emulate it:

```
A LEFT JOIN B
UNION
A RIGHT JOIN B
```

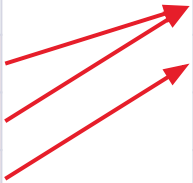
Self-join: a table joined to itself

Employee e (as employee)

EmpID	Name	SupervisorID
E1	Hoa	*NULL*
E2	Minh	E1
E3	Lan	E1
E4	Quang	E2

Employee s (as supervisor)

EmpID	Name	SupervisorID
E1	Hoa	*NULL*
E2	Minh	E1
E3	Lan	E1
E4	Quang	E2



same table, two aliases



Result

employee	supervisor
Minh	Hoa
Lan	Hoa
Quang	Minh

```
SELECT e.Name AS employee, s.Name AS supervisor
FROM Employee e
JOIN Employee s ON e.SupervisorID = s.EmpID;
```

Hoa has no supervisor
→ dropped by JOIN; use LEFT JOIN to keep her



Tutorial Q2, Q9 and Q10 pair each employee with their supervisor: a self-join. In the Sample DB: EMPLOYEE.Super_ssn → EMPLOYEE.Ssn.

Subqueries: a query inside a query



Scalar
one value
compare with =, <, >



Row
one row
compare a tuple: (a, b) = (…)

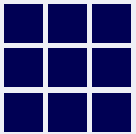


Table
many rows
use with IN, EXISTS, or in FROM

Table subquery with `IN`

```
SELECT Name FROM Student
WHERE StudentID IN (
  SELECT StudentID FROM Enrollment
  WHERE CourseID = 'ISYS2099'
);
```

Row subquery · same department and salary as Minh

```
SELECT Name, Salary FROM Employee
WHERE (Dno, Salary) = (
  SELECT Dno, Salary FROM Employee
  WHERE Name = 'Minh'
);
```



Correlated subquery: the inner query uses a column of the outer row, so it is evaluated per outer row. The next three patterns rely on it.



04

Challenging Patterns

RMIT UNIVERSITY

Pattern 1: "not in any" (the anti-join)

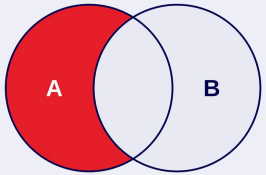
Pattern 1 · "not in any" **Which students are not enrolled in any course?** Tutorial Q3, Q5

Option A · `NOT EXISTS`

```
SELECT s.Name
FROM Student s
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollment e
  WHERE e.StudentID = s.StudentID
);
```

Option B · `LEFT JOIN ... IS NULL`

```
SELECT s.Name
FROM Student s
LEFT JOIN Enrollment e
  ON e.StudentID = s.StudentID
WHERE e.StudentID IS NULL;
```

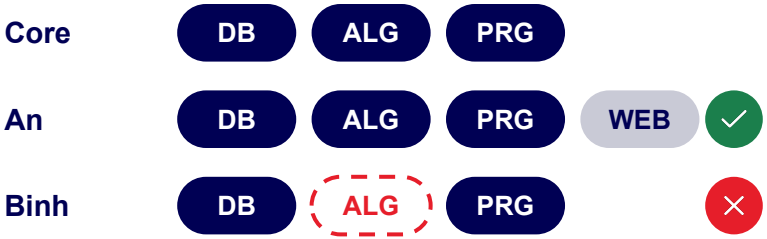


A = Student
B = Enrollment

⚠ **NOT IN trap:** if the subquery returns any NULL, **`x NOT IN (...)`** is never true, so you silently get zero rows. Prefer NOT EXISTS.

Pattern 2: "for all" (relational division)

Pattern 2 · "for all" **Which students have taken ALL core courses?** Tutorial Q6, Q12 (*)



Alternative · count them

```
SELECT e.StudentID
FROM Enrollment e
JOIN Course c ON c.CourseID = e.CourseID
WHERE c.IsCore = TRUE
GROUP BY e.StudentID
HAVING COUNT(*) = (SELECT COUNT(*)
  FROM Course WHERE IsCore = TRUE);
```

Read it as: *there is no core course that the student has not taken.*

```
SELECT s.Name
FROM Student s
WHERE NOT EXISTS (
  SELECT * FROM Course c
  WHERE c.IsCore = TRUE
    AND NOT EXISTS (
      SELECT * FROM Enrollment e
      WHERE e.StudentID = s.StudentID
        AND e.CourseID = c.CourseID));
```

Pattern 3: "the top one per group"

Pattern 3 · top per group **Who scored highest in each course?** Tutorial Q7 (*)

ISYS2099 group

S001	75
S002	82
S003	**90**

COSC2469 group

S001	68
S005	**77**

```
SELECT e.CourseID, e.StudentID, e.Score
FROM Enrollment e
WHERE e.Score = (
  SELECT MAX(e2.Score)
  FROM Enrollment e2
  WHERE e2.CourseID = e.CourseID
);
```

For each outer row, the inner query finds the top score of that row's course. Ties → all winners are returned.

MySQL 8+ alternative: `RANK() OVER (PARTITION BY CourseID ORDER BY Score DESC)`

Pattern 4: "keep the zeros"

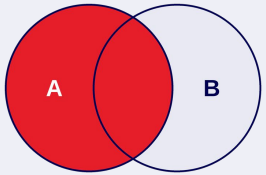
Pattern 4 · "include zero" **Count each student's courses, including zero** Tutorial Q10, Q11 (*)

✘ **Wrong · `COUNT(*)`**

```
SELECT s.StudentID,
  COUNT(*) AS courses
FROM Student s
LEFT JOIN Enrollment e
  ON e.StudentID = s.StudentID
GROUP BY s.StudentID;   -- gives 1
```

✔ **Right · `COUNT(e.CourseID)`**

```
SELECT s.StudentID,
  COUNT(e.CourseID) AS courses
FROM Student s
LEFT JOIN Enrollment e
  ON e.StudentID = s.StudentID
GROUP BY s.StudentID;   -- gives 0
```



A = Student
B = Enrollment

⚠ **Two more traps:** with a LEFT JOIN, SUM of the missing rows is NULL, not 0, so write **`COALESCE(SUM(w.Hours), 0)`**. And filter the right-hand table in ON, not WHERE, or the zero rows vanish again.

Quick check: type 1, 2 or 3 in the chat

Q1

R(StudentID, CourseID, CourseName, Score), key (StudentID, CourseID). Which normal form does it break first?

1. 1NF
2. 2NF
3. 3NF

Q2

Departments with more than 5 employees: where does COUNT(*) > 5 go?

1. `HAVING`
2. `WHERE`
3. `ORDER BY`

Q3

EMPLOYEE M:N PROJECT, with attribute hours. How many tables after mapping?

1. 2
2. 4
3. 3

Tutorial toolkit: which tool for which question?

Q	What it asks	Tools to reach for
1	Research employees working ≥ 10 h/week on some project	JOIN 3 tables + WHERE
2	Employees supervised by managers from Research	Self-join or nested IN
3	Department managers not working on any project	Anti-join · NOT EXISTS (pattern 1)
4	Each project with total weekly hours	JOIN + GROUP BY + SUM
5	Employees not on any project in Houston	Anti-join with a condition inside (pattern 1)
6 *	Employees on ALL projects in Houston	Division · double NOT EXISTS or COUNT (pattern 2)
7 *	Highest-paid employee(s) in each department	Top per group · correlated subquery (pattern 3)
8	Departments with at least 3 employees	GROUP BY + HAVING COUNT(*)
9	Employees older than their direct supervisor	Self-join + compare birth dates
10 *	Employees on more projects than their direct supervisor	Self-join + counts that keep zeros (pattern 4)
11 *	Per department: weekly hours on other departments' projects	LEFT JOIN + COALESCE(SUM, 0) (pattern 4)
12 *	Employees on every project their own department controls	Division, NOT EXISTS version (pattern 2)
Ex 2	Normalize Order to 1NF, 2NF, 3NF	The three-step recipe from Part 03

* = challenge question · patterns 1 to 4 are on the previous slides

Tutorial this week: get set up

1

Install MySQL

Windows or macOS · Canvas guide, 30–45 min

2

Load the sample DB

Run company.sql: 6 tables + data

3

Study the diagram

Use the Sample DB diagram: real tables, keys and column names

4

Solve

Exercise 1: 12 SQL queries · Exercise 2: normalize Order

First commands to run

```
SHOW TABLES;                    -- what's here?
DESCRIBE <table>;               -- columns & keys
SELECT * FROM <table> LIMIT 5;  -- peek at data
```

Canvas → W1 - Installing MySQL Server and Workbench: install guide, Sample DB and lab login details.

Work smart

- Build queries in steps: FROM/JOIN → check rows → WHERE → GROUP BY
- Check results by hand on a few rows
- Workbench: read timeout 600 s, untick Safe Updates
- Stuck? Post in the Week 1 Discussion

Key takeaways

ER diagrams capture what to store

entities, attributes, relationships, cardinality, participation

Six mapping rules turn ER into tables

keys and foreign keys link them together

Normalize one step at a time

atomic → the whole key → nothing but the key

Think in SQL's logical order

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Next week · W2

Database indexing

Your query is instant on 1,000 rows. What happens at 10 million?

Also: how database applications are structured

Thank you

Questions? Type them in the chat

RMIT UNIVERSITY



Copyright © 2023 RMIT University Vietnam